OilShield — India Energy Resilience Command Center

Problem Statement: PS 2 — AI-Driven Energy Supply Chain Resilience for Import-Dependent Economies

From crisis firefighting to anticipatory decisions

Executive Summary

OilShield is an integrated, single-screen command center that turns the chaos of a crude-oil supply shock into a structured, auditable decision in seconds. It fuses three connected modules — a **Live Risk Radar** that reads the geopolitical signal environment, a **Disruption Scenario Simulator** that projects the economic cascade of a corridor closure or production cut, and an **Adaptive Procurement Recommender** that ranks alternative supply options — behind a one-click **signal-to-recommendation pipeline** that measures and displays its own end-to-end latency. The system is built on a "deterministic core, probabilistic edges" principle: all risk, impact, and procurement math is pure, reproducible Python, while only the extraction of structure from unstructured news touches an LLM (with a deterministic fallback that guarantees the pipeline always completes). The result is a product that looks and behaves like production SaaS, degrades gracefully to fully offline simulated data, and traces every number it shows back to the evidence that produced it — exactly the transparency an import-dependent economy needs to move from reactive firefighting to anticipatory planning.

Table of Contents

---

1. The Problem & Why Now

India imports roughly **88% of the crude oil it consumes**, making it one of the most import-dependent major economies in the world. A large share of those barrels moves through a handful of maritime chokepoints — an estimated **40–45% of India's crude flows transit the Strait of Hormuz** alone. Against that exposure, India holds only about **9.5 days of strategic petroleum reserves** as a direct national buffer, a thin cushion when a corridor can close overnight.

The threat is not hypothetical. During the **2025 US–Iran standoff, Brent crude spiked roughly 8% in a single trading day**. Persistent **Red Sea / Houthi shipping disruptions** have forced tankers onto longer, costlier routes and reshaped freight economics for months at a time. Each of these events compresses the window in which a country must decide how to respond.

Yet the tools most planners rely on are **static** — spreadsheets, periodic PDF reports, and dashboards that describe the past rather than project the consequences of a live event. When a shock lands, teams scramble to reconcile news, model impact, and evaluate alternative suppliers manually, losing days precisely when days matter most.

> **The core business case:** McKinsey analysis indicates that economies **without** smart, responsive tooling took **47 extra days to recover** from oil shocks compared with those that had it. Forty-seven days of avoidable disruption is the gap OilShield is designed to close.

Why now: cheap, fast LLMs can finally read the unstructured signal environment in real time, and a transparent deterministic model layer can turn that reading into auditable projections — the two halves of anticipatory decision-making are, for the first time, both practical inside a single lightweight application.

---

2. Solution Overview

OilShield delivers three connected modules inside one dark-mode command center, plus an end-to-end pipeline that stitches them together:

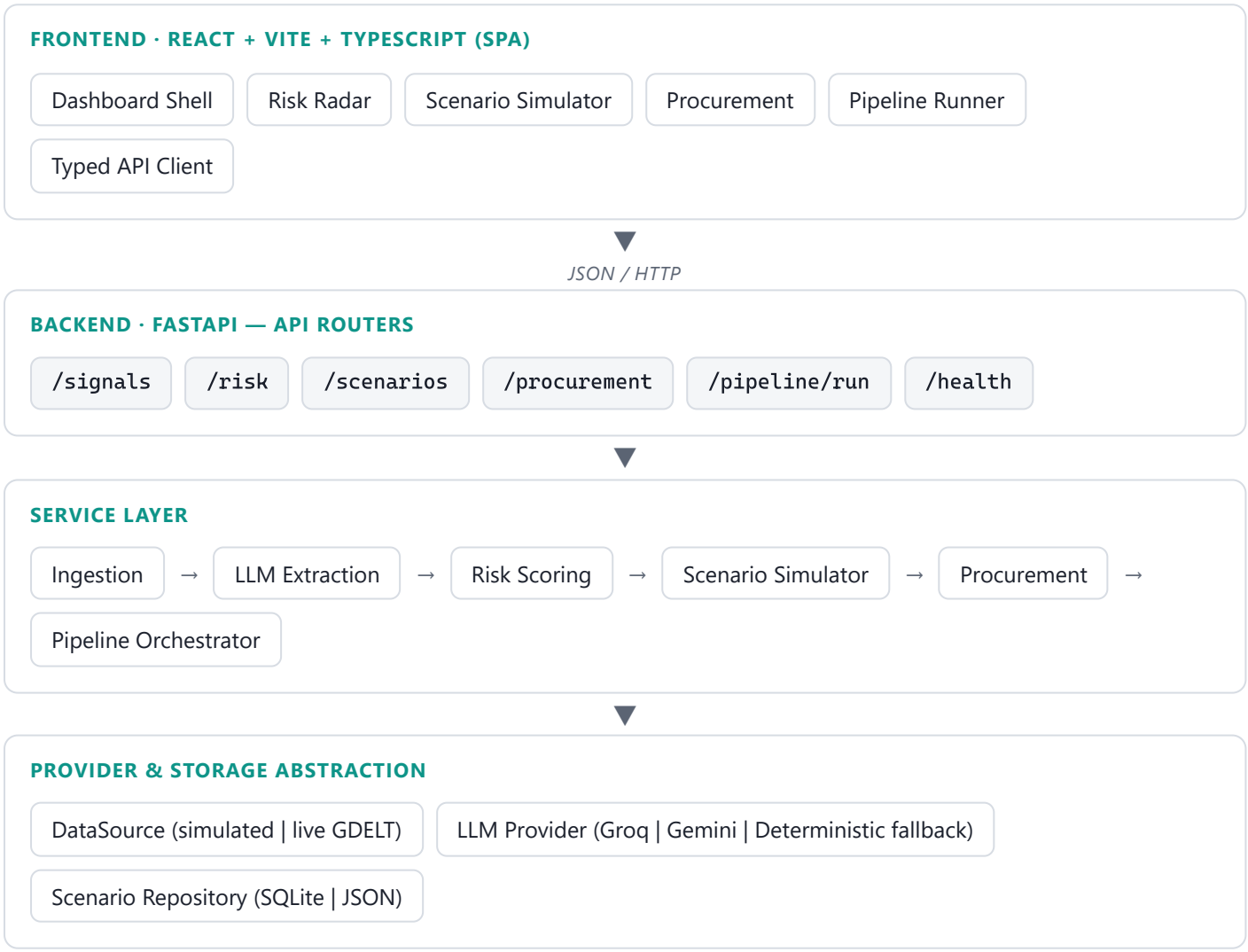
- **Live Risk Radar** — ingests news and signals, extracts structured risk from them, aggregates a per-corridor and per-country risk score, and displays it on a map and a ranked list with full evidence traceability.
- **Disruption Scenario Simulator** — lets a planner configure explicit, testable assumptions (corridor closure, production cut, duration, import share, starting reserves) and projects a deterministic day-by-day cascade across refinery run-rate, fuel price, reserve days-of-cover, and GDP. Scenarios can be saved and reloaded.
- **Adaptive Procurement Recommender** — scores and ranks alternative supply options using a transparent weighted formula, filters out incompatible grades, and attaches a plain-language rationale to each recommendation.

Tying them together is a **one-click "signal → recommendation" pipeline**: it runs ingestion → risk scoring → scenario impact → procurement in sequence, surfaces linked actions when a corridor enters the high-risk band, and **displays its own measured latency** so the speed of the decision loop is visible on screen.

---

3. How It Works — Architecture

OilShield is a two-tier application: a React single-page app talking to a FastAPI backend over JSON/HTTP. The backend is organized into a thin API layer, a service layer (the three modules plus a pipeline orchestrator), and an abstraction layer over external providers (data feeds, LLM) and storage. Every external dependency sits behind an interface, so the same business logic runs whether data is live or simulated.



Deterministic core (scoring, simulation, procurement) · probabilistic edges (LLM extraction, live feeds) with graceful fallback to simulated data.

Deterministic core, probabilistic edges

The defining architectural choice is where non-determinism is allowed to live. **Risk aggregation, scenario impact, and procurement scoring are pure deterministic functions** of their typed inputs — given the same inputs they always return the same outputs. The **only** place an LLM is used is to extract structure (target corridor, risk category, severity) from free-text news, and even that has a deterministic fallback

extractor built from the signal's own severity hint. This makes the system testable, reproducible, and safe to demo live: LLM variability can never change the numbers a judge sees.

Live-vs-simulated provenance with graceful degradation

Every external dependency ships with a bundled, realistic fallback and a visible `Data_Source_Mode` badge. **Simulated data is the default**, so the demo runs fully offline out of the box. Optionally, live mode enables **Groq / Gemini** for extraction and **GDELT news** for signals. If a live source or the LLM fails or times out, the service transparently loads bundled data and flips the provenance badge — the pipeline never goes dark because a Wi-Fi connection dropped on stage. Genuinely malformed data, by contrast, fails loudly rather than being silently coerced, so bugs stay visible in development.

---

4. Feature Detail Per Module

4.1 Live Risk Radar

The Risk Radar reads the signal environment and turns it into a spatial, ranked, evidence-backed risk picture:

- **Ingestion → extraction.** Raw news/signals are normalized into typed `Signal` records, then passed to the LLM extractor (**Groq primary, Gemini secondary**), which produces a structured `ExtractedSignal` (target corridor/country, risk category, severity 0–100). On any LLM failure or timeout, a **deterministic fallback** derives severity from the signal's raw hint, so extraction always yields a usable result. Signals that cannot be mapped to a known target are labeled *unclassified* and excluded from scoring.
- **Aggregation.** Classified signals are combined into a per-target risk score using a **noisy-OR style aggregation**, which behaves intuitively: additional corroborating signals push the score up without ever exceeding the bounded [0, 100] range.
- **Bands.** Each score is banded — **low (0–33), elevated (34–66), high (67–100)** — a total classification with no gaps or overlaps at the boundaries.
- **Visualization & traceability.** Corridors render as colored polylines on the map by band, alongside a ranked list ordered highest-to-lowest. Selecting any target opens its contributing signals with **source and timestamp**, so every score traces directly back to the evidence that produced it.

4.2 Disruption Scenario Simulator

The simulator makes its assumptions explicit and its math auditable. Every numeric assumption is surfaced in the UI and applied through documented formulas — judges can see exactly why a number moved.

Adjustable, range-validated assumptions:

Assumption	Meaning
`corridor_closure_pct`	Fraction of a corridor's throughput lost (0–100%)
`corridor_import_share`	Share of India's crude imports through this corridor (0–1)
`production_cut_kbd`	Supply removed in thousand barrels/day (OPEC+ scenarios)
`duration_days`	Scenario horizon (1–180 days)
`spr_start_days`	Starting strategic-reserve days-of-cover

Documented deterministic cascade. A single derived supply shock drives the whole timeline:

```
supply_loss_fraction = clamp(
    corridor_import_share * (corridor_closure_pct / 100)
      + production_cut_kbd / TOTAL_IMPORT_KBD, 0, 1)

refinery_run_rate_pct(day) = clamp(100 − K_REF * supply_loss_fraction * 100, 0, 100)
fuel_price_index(day)      = 100 * (1 + K_PRICE * supply_loss_fraction)
spr_days_of_cover(day)     = max(0, spr_start_days − day * supply_loss_fraction / DRAWDOWN_
gdp_index(day)             = 100 * (1 − K_GDP * supply_loss_fraction * (day / duration_days
```

Documented constants (centralized, tunable): `TOTAL_IMPORT_KBD = 5000`, `K_REF = 0.8`, `K_PRICE = 1.5`, `K_GDP = 0.5`, `DRAWDOWN_DIVISOR = 2.0`. Because all coefficients are non-negative, the model's invariants hold *by construction*: higher closure never *raises* reserve days-of-cover, **SPR days-of-cover is clamped at ≥ 0**, run rate stays in [0, 100], and a zero shock leaves every index flat at 100. Scenarios can be **saved and reloaded** with a version-checked round-trip.

4.3 Adaptive Procurement Recommender

The recommender ranks alternative supply options with a transparent weighted score. Each attribute is normalized to [0, 1] where 1 is best, then combined with fixed weights that sum to 1 and scaled to [0, 100]:

```
price_score   = clamp((PRICE_CEILING − spot_price) / (PRICE_CEILING − PRICE_FLOOR), 0, 1)
avail_score   = tanker_availability          # higher is better
congest_score = 1 − port_congestion          # higher congestion lowers the score
compat_score  = grade_compatibility          # higher is better

recommendation_score = 100 * (W_PRICE*price_score + W_AVAIL*avail_score
                            + W_CONGEST*congest_score + W_COMPAT*compat_score)
```

Weight	Value	Normalization / Rule
`W_PRICE`	0.35	`PRICE_FLOOR = 40`, `PRICE_CEILING = 120` (USD/bbl)
`W_AVAIL`	0.20	tanker availability, 0–1
`W_CONGEST`	0.15	port congestion inverted to a score
`W_COMPAT`	0.30	grade compatibility, 0–1

Options whose grade compatibility falls below `MIN_COMPAT = 0.4` are excluded before ranking. Because every weight is non-negative and each sub-score is monotone in its attribute, the ranking is intuitive — cheaper, more available, less congested, more compatible options rank higher. Results are returned sorted descending, each with a **plain-language rationale** explaining why it placed where it did.

---

5. Judging Alignment

Innovation (25%)

OilShield's novelty is the **integration** and the **discipline**, not a single flashy model. It closes the full loop from a live geopolitical signal to a ranked procurement action in one screen, and it does so with a principled split — *deterministic core, probabilistic edges* — that most "AI dashboards" lack. The LLM is used precisely where it excels (reading messy text) and nowhere near the numbers, which is a genuinely thoughtful application of AI to a high-stakes domain.

Business Impact (25%)

The value proposition is quantified and specific: economies without smart response tooling took **47 extra days** to recover from oil shocks. OilShield attacks that gap directly by compressing the sense-model-decide loop from days of manual reconciliation into a single measured pipeline run. For an economy importing 88% of its crude with ~9.5 days of reserve cover, faster, auditable decisions translate into avoided disruption cost.

Technical Excellence (20%)

Typed models end-to-end (Pydantic on the backend, mirrored TypeScript on the frontend), a clean provider/interface abstraction, a consistent JSON error envelope, and a serious test posture: **46 backend tests pass across unit, integration, and performance suites**, the frontend **type-checks and builds cleanly**, and the design formally specifies **23 correctness properties** for property-based testing.

Scalability (15%)

Every scaling seam is a real interface. The `DataSource` and `LLM` providers can move from bundled JSON and free-tier models to paid AIS/commodity feeds and hosted models with **no service-code changes**.

Services are stateless and pure where possible, so the backend scales horizontally without session affinity. The `ScenarioRepository` interface hides SQLite, making a Postgres swap a one-file change.

User Experience (15%)

A single dark-mode command center presents all three modules without navigation friction, with a map, timeline charts, status badges, an animated pipeline stepper, and a prominent latency readout. Each module owns its own loading/error/data states so a failure in one never blanks the others, and the provenance banner keeps data source honesty always visible.

---

6. Engineering Quality

- **Typed models end-to-end.** Data shapes are defined once as Pydantic models on the backend and mirrored as TypeScript types on the frontend, with validators enforcing field ranges so invalid states are unrepresentable. Shape errors surface at compile time rather than in a live demo.
- **Provider / interface abstraction as swap seams.** `DataSourceProvider`, `LLMProvider`, and `ScenarioRepository` are protocols with interchangeable concrete implementations (`LiveDataSource` / `SimulatedDataSource`; `GroqProvider` / `GeminiProvider` / `DeterministicExtractor`; `SqliteScenarioRepository` / `JsonFileScenarioRepository`). Services depend on the interfaces, never the SDKs.
- **Consistent JSON error envelope.** A typed error hierarchy (`DataSourceError`, `LLMError`, `NormalizationError`, `ValidationError`, `ScenarioLoadError`) maps through a FastAPI exception handler to a uniform `{ "error": { "module", "message", "code" } }` response, so the frontend can render module-scoped errors predictably.
- **Testing. 46 backend tests pass** across unit, integration, and performance suites; the frontend **type-checks and builds cleanly**. End-to-end pipeline latency measures in the **low single-digit milliseconds in simulated mode** (comfortably inside the 15-second budget). Beyond example tests, the design specifies **23 correctness properties** — each intended to be verified by a single property-based test running 100+ iterations (Hypothesis on the backend, fast-check on the frontend). This is both a current rigor signal and a clear testing roadmap.

---

7. Honest Limitations & Assumptions

We are deliberately transparent about what OilShield is and is not:

- **Scenario and risk numbers are illustrative, auditable demo models — not calibrated forecasts.** The cascade constants (`K_REF`, `K_PRICE`, `K_GDP`, `DRAWDOWN_DIVISOR`) and the procurement weights are documented, reasonable defaults chosen to sit inside realistic ranges. They demonstrate the mechanism and are fully auditable, but they have not been fit to historical data.

- **Simulated data is the default.** The bundled JSON datasets (signals, corridors, routes, procurement options) drive the offline demo. Live mode (Groq/Gemini extraction + GDELT news) is optional and degrades gracefully back to simulated on any failure.
- **The risk aggregation and impact cascade are transparent rules, not trained models.** This is intentional for testability and reproducibility, but it means the outputs reflect the encoded assumptions rather than a learned relationship.
- **Authentication and CORS are open for the demo.** The API currently allows all origins and has no auth layer. This is appropriate for a hackathon demo but must be closed before any real deployment.
- **Coverage is a chokepoint/corridor subset**, not an exhaustive model of every route, grade, or supplier.

---

8. Roadmap

1. **Calibrate the models** — fit the cascade constants and risk aggregation weights against historical oil-shock data so projections become defensible forecasts rather than illustrative demos.
2. **Integrate real feeds** — connect live AIS vessel tracking, port-congestion data, and commodity/spot-price feeds through the existing `DataSource` interface.
3. **Alerting** — push notifications and thresholds when a corridor crosses into the high band or a scenario breaches a reserve floor.
4. **Multi-country** — generalize beyond India to other import-dependent economies by parameterizing import shares, reserves, and corridors.
5. **Production hardening** — add authentication and RBAC, tighten CORS to known origins, add rate limiting and observability, and move storage from SQLite to a managed database.

---

9. How to Run

Concise Windows steps (from the repository root):

Backend (FastAPI):

```
cd oilshield\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e .
uvicorn app.main:app --reload
# API docs at http://localhost:8000/docs ; health at http://localhost:8000/health
```

Run the backend tests:

```
cd oilshield\backend
pytest
```

Frontend (React + Vite):

```
cd oilshield\frontend
npm install
npm run dev
# open the printed local URL (typically http://localhost:5173)
```

The app runs fully offline in simulated mode by default. To enable live mode, provide `GROQ_API_KEY` and/or `GEMINI_API_KEY` in the backend environment; if keys are absent or a call fails, OilShield automatically falls back to simulated data.

Repository: `<INSERT GITHUB URL>`

---

10. Closing

OilShield turns an oil shock from a 47-day scramble into a 15-second decision.